

Project Plan – AI SDLC Terminal

Project Information

Field	Details
Project Name	AI SDLC Terminal
Project Owner	Chinmay Shete
Project Type	AI + DevOps + Automation
Version	v1.0
Status	Planning
Start Date	May 2026
Expected Completion	July 2026

1. Project Summary

The AI SDLC Terminal project aims to develop an intelligent command-line platform capable of automating Software Development Life Cycle (SDLC) activities using AI agents and DevOps integrations.

The platform will centralize multiple development and operational workflows into a single AI-driven terminal capable of interacting with tools such as Jira, GitHub, Jenkins, Docker, Kubernetes, and security scanners.

The objective is to improve developer productivity, automate repetitive operations, reduce manual intervention, and enable intelligent workflow orchestration.

2. Problem Statement

Modern SDLC workflows require engineers to switch between multiple tools for project management, CI/CD operations, deployment, monitoring, and security scanning.

This leads to:

- Increased operational complexity
- Manual coordination overhead
- Delayed deployments
- Lack of centralized automation

- Reduced engineering productivity
- Inefficient issue tracking and monitoring

The AI SDLC Terminal project addresses these challenges by introducing a unified AI-powered automation interface.

3. Project Objectives

Primary Objectives

- Build an AI-powered CLI terminal
- Automate SDLC workflows
- Integrate DevOps and AI systems
- Enable intelligent agent orchestration
- Support CI/CD automation
- Automate infrastructure management
- Generate AI-assisted operational insights

Secondary Objectives

- Improve deployment speed
 - Reduce manual intervention
 - Enable scalable architecture
 - Enhance security automation
 - Improve monitoring and observability
-

4. Scope

In Scope

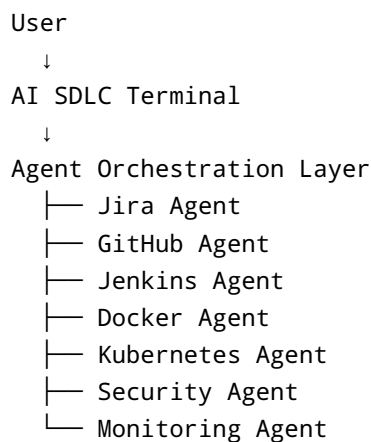
- Python-based CLI terminal
- LangGraph-based agent workflows
- Jira integration
- GitHub integration
- Jenkins pipeline automation
- Docker container management
- Kubernetes orchestration
- Security scanning integration
- AI workflow execution
- Logging and monitoring

Out of Scope

- Native mobile application
 - Desktop GUI application
 - Multi-cloud provisioning
 - Billing automation
 - Enterprise IAM implementation
-

5. System Architecture

High-Level Architecture



Core Components

CLI Engine

Handles user interaction and command execution.

Agent Layer

Responsible for orchestration of intelligent workflows.

Integration Layer

Communicates with external DevOps platforms.

AI Processing Layer

Executes LLM-driven reasoning and automation.

Monitoring Layer

Tracks logs, metrics, and execution events.

6. Technology Stack

Layer	Technology
Programming Language	Python
AI Framework	LangGraph
LLM Integration	OpenAI API
CLI Framework	Typer / Rich
Project Management	Jira
Source Control	GitHub
CI/CD	Jenkins
Containerization	Docker
Orchestration	Kubernetes
Security Scanning	Trivy / Semgrep
Logging	ELK Stack
Monitoring	Prometheus + Grafana
Documentation	Confluence

7. Functional Requirements

User Management

- User authentication
- Role-based access
- Session handling

AI Automation

- Natural language command execution
- AI-driven task recommendations
- Multi-agent workflow execution

DevOps Integration

- Trigger Jenkins pipelines
- Create Jira tickets
- Manage GitHub repositories
- Deploy Docker containers
- Manage Kubernetes workloads

Security Automation

- Run SAST scans
- Run dependency scans
- Generate vulnerability reports

Monitoring

- View execution logs
 - Track deployment status
 - Generate operational metrics
-

8. Non-Functional Requirements

Requirement	Description
Performance	Fast command execution
Scalability	Support multiple agents
Reliability	Retry and recovery mechanisms
Security	Secure API communication
Maintainability	Modular architecture
Observability	Centralized logging and monitoring

9. Project Phases

Phase	Description	Duration
Phase 1	Requirement Gathering	Week 1
Phase 2	Architecture Design	Week 1
Phase 3	CLI Development	Week 2

Phase	Description	Duration
Phase 4	AI Agent Integration	Week 3
Phase 5	DevOps Integration	Week 4
Phase 6	Security Automation	Week 5
Phase 7	Testing & Validation	Week 6
Phase 8	Deployment & Monitoring	Week 7

10. Timeline

Week	Activity
Week 1	Project Planning & Architecture
Week 2	Python CLI Development
Week 3	AI Agent Implementation
Week 4	Jira & GitHub Integration
Week 5	Jenkins & Docker Integration
Week 6	Kubernetes & Security Scanning
Week 7	Testing & Deployment
Week 8	Monitoring & Optimization

11. Deliverables

Primary Deliverables

- AI SDLC Terminal CLI
- AI Agent Orchestration Engine
- DevOps Integration Modules
- Security Scanning Engine
- Deployment Pipelines
- Monitoring Dashboard
- Documentation

Documentation Deliverables

- Architecture Documentation
 - API Documentation
 - User Guide
 - Deployment Guide
 - Troubleshooting Guide
-

12. Risk Management

Risk	Impact	Probability	Mitigation
API Failure	High	Medium	Retry mechanisms
Security Vulnerabilities	High	Medium	Automated scanning
Integration Failure	Medium	Medium	Validation testing
LLM Latency	Medium	High	Caching and optimization
Infrastructure Downtime	High	Low	Monitoring and alerts

13. Dependencies

External Dependencies

- OpenAI API
- Jira Cloud APIs
- GitHub APIs
- Jenkins Server
- Docker Engine
- Kubernetes Cluster

Internal Dependencies

- Python runtime
 - Agent orchestration engine
 - Logging infrastructure
-

14. Team Responsibilities

Role	Responsibility
Project Owner	Project management
Backend Developer	CLI and API development
AI Engineer	Agent orchestration
DevOps Engineer	CI/CD and deployment
Security Engineer	Security scanning
QA Engineer	Testing and validation

15. Success Criteria

The project will be considered successful if:

- All core integrations work successfully
- AI workflows execute correctly
- CI/CD automation is operational
- Security scans execute automatically
- Deployment pipelines work end-to-end
- Monitoring and logging are functional
- Documentation is complete

16. Future Enhancements

Planned Improvements

- Voice-enabled commands
- Multi-cloud deployment support
- Advanced AI reasoning workflows
- Web dashboard
- Plugin marketplace
- Auto-remediation capabilities

17. Jira Integration

Planned Jira Workflows

- Automatic ticket creation
 - Sprint management integration
 - Issue tracking automation
 - Release tracking
 - Story point analysis
-

18. CI/CD Workflow

```
Code Commit
  ↓
GitHub Push
  ↓
Jenkins Pipeline Trigger
  ↓
Build & Test
  ↓
Security Scan
  ↓
Docker Build
  ↓
Kubernetes Deployment
  ↓
Monitoring & Alerts
```

19. Monitoring & Logging

Monitoring Features

- Pipeline monitoring
- Deployment monitoring
- Resource utilization tracking
- AI workflow monitoring
- Error alerting

Logging Features

- Centralized logs
 - Audit trails
 - Command execution logs
 - Security event logging
-

20. Approval Section

Name	Role	Status
Chinmay Shete	Project Owner	Pending
DevOps Lead	Reviewer	Pending
Architecture Team	Reviewer	Pending

21. References

- Jira Documentation
 - Jenkins Documentation
 - Kubernetes Documentation
 - LangGraph Documentation
 - OpenAI API Documentation
 - Docker Documentation
-

22. Conclusion

The AI SDLC Terminal project aims to transform traditional SDLC workflows by introducing AI-powered automation, intelligent orchestration, and centralized DevOps management.

The platform will improve engineering productivity, reduce operational complexity, and provide scalable enterprise automation capabilities.